

Spherical Easel Tours



Team:

Wyatt Watson, Sam Rios, Ben Turner, Jason Bos

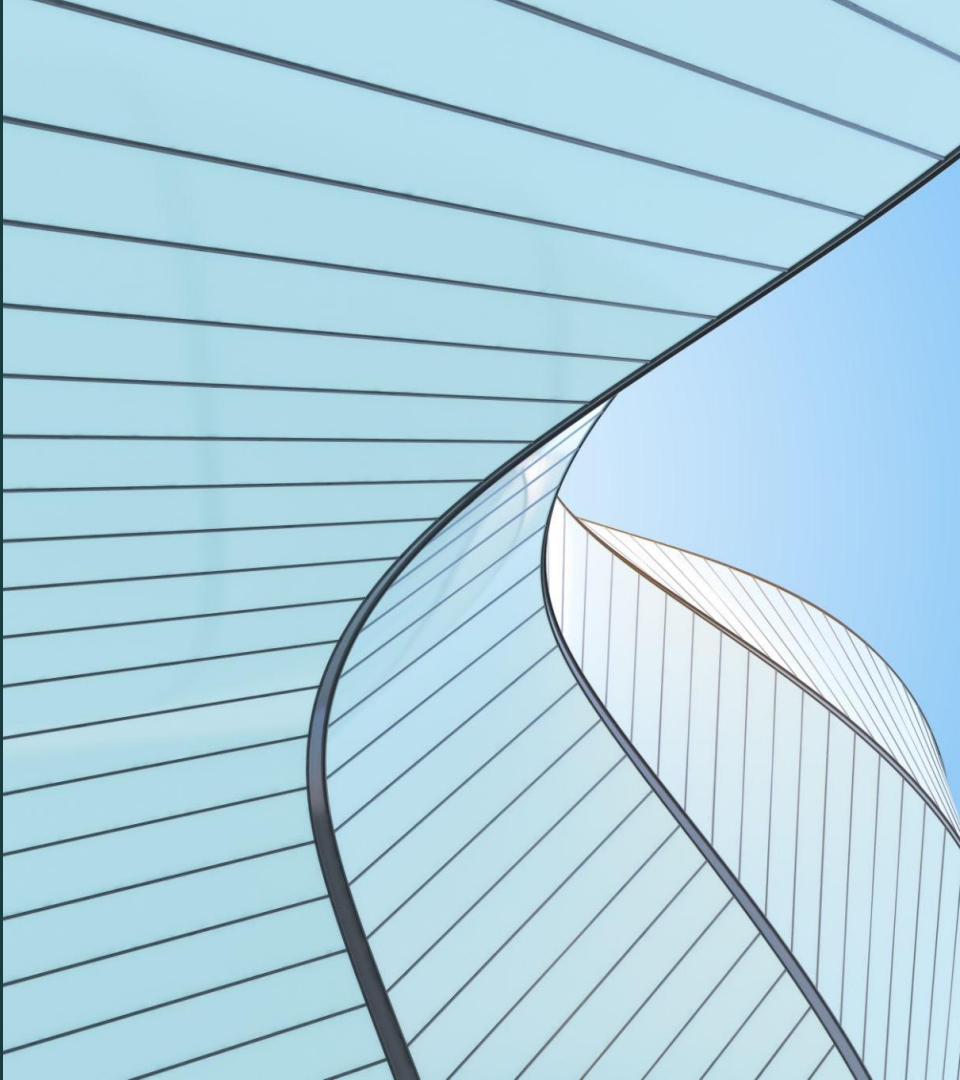


Table of Contents

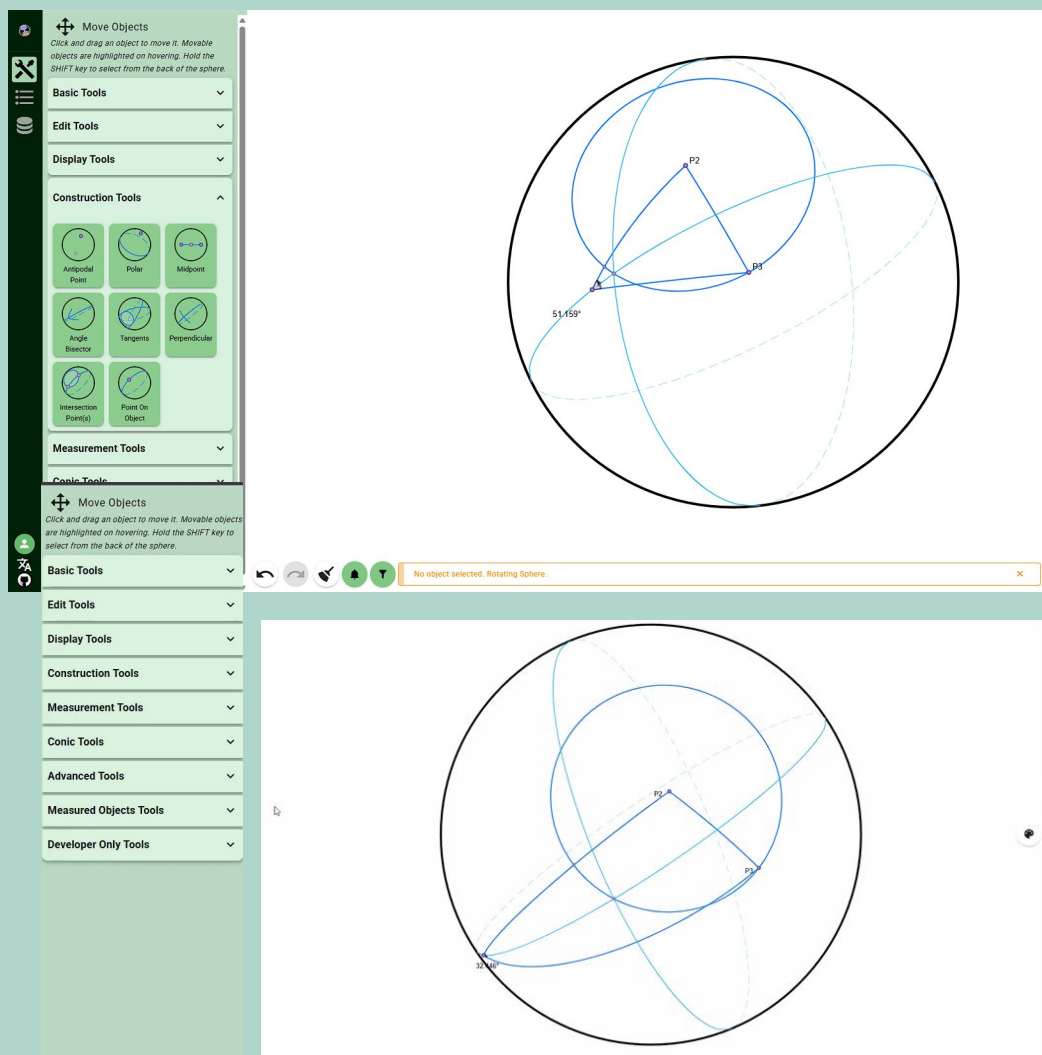
- Overview
- Toolset (Languages, Libraries, APIs)
- Repo Structure
- System Organization
- Development
- Non-Trivial Requirements
- Work Policies
- Testing
- Deliverance and Milestones

Overview

Spherical Easel is a web app that specializes in spherical geometry. Spherical geometry is a very complex topic (and interesting too!), and with it comes this tool with many great features.

First time users can be overwhelmed with all the features. Our goal in this project is to create tutorials for some features, then streamline our system of making tutorials in the backend, and if all goes well, into a UI based system.

<https://easelgeo.app/>



Toolset

4

- **Languages**
 - Typescript
- **Environment**
 - Bun (Node.js Alt)
- **Libraries**
 - Vue.js & Vuetify
 - Vue l18n
 - Pinia (States Library)
- **Database**
 - Firebase (Possible Feature)



Repo Structure

Main Branch

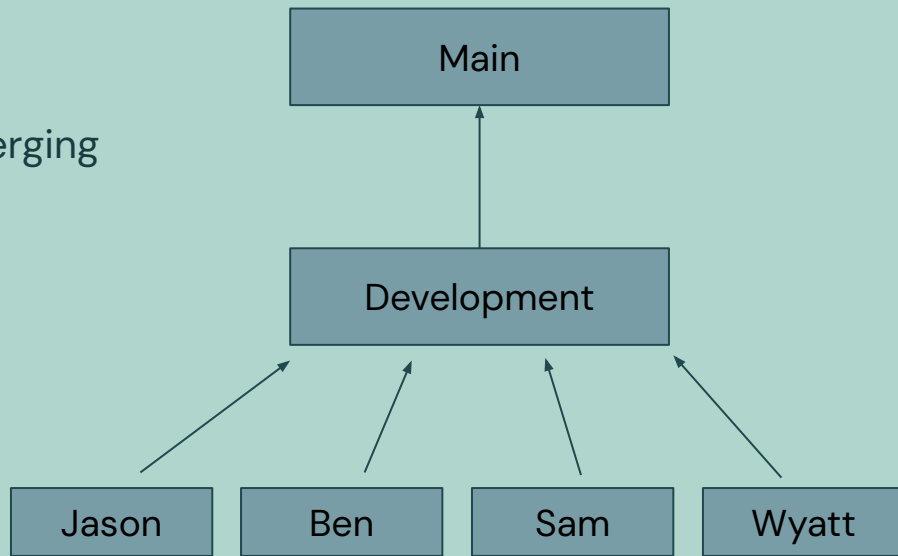
- Stable version of the project
- Contains finished and functional code
- Requires team review and testing before merging
- Most stable version for releases and demos

Development Branch

- Shared integration branch for active work
- Combines features from team members
- Used for team testing and demonstrations
- Keeps unfinished work out of main

Personal Feature Branches

- Created from the development branch
- Used for individual development and local testing
- Each branch is tied to a Linear issue
- Merged into development through a pull request



System Organization

The tour system will be directly integrated within the Spherical Easel application. Therefore, for the tours themselves, we do not need an outside server to manage them.

The tour system itself will include two parts: the reusable tour framework and individual tour definitions.

This route allows for future developers to follow the framework to make the tour making process streamlined

Tour Framework:

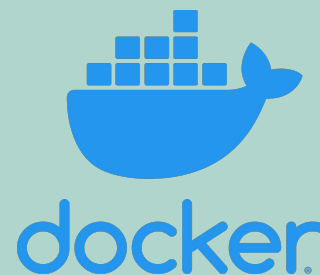
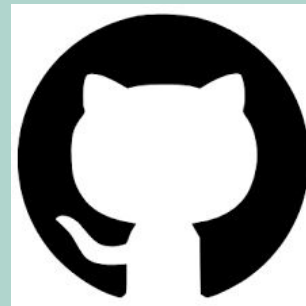
- Current tour, current step, popup display, UI highlighting, navigation, and completion conditions

Individual tour definitions:

- Describes the steps of the tour
 - Message, target interface element, and expected user action

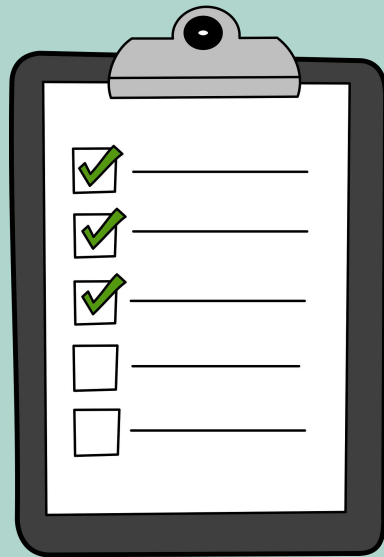
Development

- VSCode
- GitHub
- Bun
- Docker



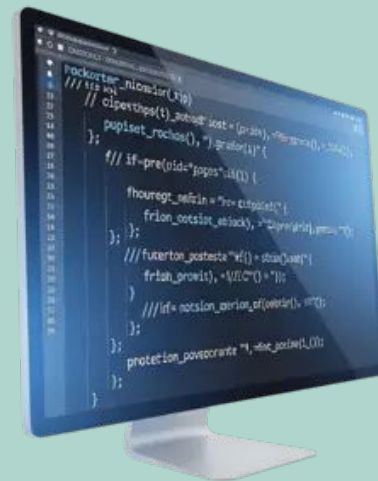
Non-Trivial Requirements

- **Create a tour system that takes user input**
 - Instead of a “Next” button, have user complete on screen instruction
 - Makes it more engaging for the user.
- **Reusability of tours framework**
 - Ensuring that the framework can be reused
 - Goal: Backend tour development is streamlined
 - Extra: Implementing it with UI based elements.



Work Policies

- **Sponsor contact and project management:**
 - Wyatt Watson
- **Presentation and Documentation deliverables:**
 - Wyatt Watson
 - Sam Rios
- **Development and Testing:**
 - Wyatt Watson
 - Sam Rios
 - Ben Turner
 - Jason Bos



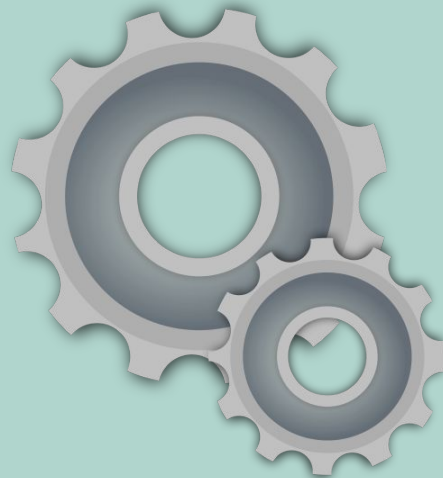
Testing

- **Individual Testing**

- Each developer will test their own code
- Unit testing in Vue
- Visually inspecting UI as we develop

- **Pull Requests**

- Developer will inform team of their new code
- Code will be reviewed by team
- At least one member must approve before merging.



Deliverance and Milestones

- **Sprint 1**

- Research
 - Codebase exploration
 - GitHub workflow
- Tool setup
 - Set up local dev environments
 - Tour prototypes/ideas

- **Sprint 2**

- Develop tour structure and framework
- First working tour'

- **Sprint 3**

- User action directions and completion detection
- Additional tours and possible reusable dynamic tour

- **Sprint 4**

- UX testing and refinement
- Core feature completion and stable demo

- **Sprint 5**

- Final testing, documentation, and report
- Demo video, presentation, and final client review